



Institut National des Postes et Télécommunications
SUD — Systèmes Ubiquitaires et Distribués : Cloud & IoT



MyHeart Healthcare System

Système de Gestion des Soins de Santé

Architecture Microservices & Déploiement à l'aide de Docker

Informations du Projet

 École :	INPT
 Filière :	SUD
 Étudiante :	Nasr Houda
 Encadrant :	M. Hassan FARSI
 Module :	Architecture Orientée Services (SOA)
 Date :	1 ^{er} mars 2026



Table des matières

1	Introduction	2
1.1	Contexte du projet	2
1.2	Problématique	2
1.3	Objectifs du projet	2
1.3.1	Objectifs fonctionnels	2
1.3.2	Objectifs techniques	2
1.4	Solution proposée	3
2	Partie I : Conception et Architecture	4
2.1	Identification des microservices	4
2.2	Architecture globale	5
2.3	Communication entre services	6
2.4	Diagramme entité-relation (ERD)	6
2.5	Structure des bases de données	6
2.5.1	Bases PostgreSQL (services Java / Spring Boot)	7
2.5.2	Bases MongoDB (services Node.js)	7
2.6	Architecture de déploiement Docker	8
2.7	Tableau récapitulatif des ports	8
2.8	Stack technologique	9
3	Partie II : Implémentation et Déploiement	10
3.1	Flux de navigation dans l'application	10
3.2	Diagramme de séquence — Prise de rendez-vous	10
4	Interfaces Utilisateur	11
4.1	Page de connexion et sélection de rôle	11
4.2	Espace Patient	11
4.3	Espace Médecin	12
4.4	Espace Pharmacie	12
4.5	Espace Laboratoire	13
4.6	Espace Réception	13
4.7	Espace Caisse	14
5	Tests et Validation	15
5.1	Tests des API avec curl	15
5.2	Vérification des bases de données	16
6	Déploiement avec Docker	17
6.1	Fichier docker-compose.yml	17
6.2	Commandes de déploiement	20
7	Conclusion	22
7.1	Bilan du projet	22
7.2	Difficultés rencontrées	22
7.3	Améliorations futures	22
	Annexes	23

1 Introduction

1.1 Contexte du projet

Dans le cadre du module « **Architecture Orientée Services (SOA) et Développement Web** » dispensé par **M. Hassan FARSI**, ce mini-projet a pour objectif de nous offrir une expérience pratique du paradigme orienté services à travers l'adoption d'une architecture microservices.

Ce projet, intitulé **MyHeart Healthcare System**, consiste à concevoir et développer un système de gestion des soins de santé permettant d'améliorer la prise en charge des patients, l'efficacité du personnel médical et la coordination entre les différents services d'un établissement de santé.

1.2 Problématique

Un vaste réseau hospitalier fait face à des défis majeurs pour gérer efficacement le nombre croissant de patients, les rendez-vous, la facturation et les dossiers médicaux. Le système actuel éprouve des difficultés pour :

- **Planifier les rendez-vous** : Les patients rencontrent des difficultés pour trouver des créneaux disponibles.
- **Traiter les paiements** : La facturation est lente et sujette aux erreurs.
- **Gérer les prescriptions** : Les ordonnances sont souvent perdues ou mal interprétées.
- **Gérer les rapports de laboratoire** : Les résultats d'analyses sont difficilement accessibles.
- **Centraliser les dossiers médicaux** : Les informations des patients sont dispersées.

1.3 Objectifs du projet

1.3.1 Objectifs fonctionnels

- Faciliter la prise de rendez-vous en ligne
- Centraliser les dossiers médicaux électroniques (EHR)
- Gérer les prescriptions électroniquement
- Traiter et consulter les analyses de laboratoire
- Automatiser la facturation
- Offrir une interface unifiée et personnalisée par rôle

1.3.2 Objectifs techniques

- Adopter une architecture microservices
- Assurer la scalabilité horizontale du système
- Garantir la fiabilité et la sécurité des données
- Conteneuriser l'application avec Docker et Docker Compose
- Assurer l'interopérabilité et la communication entre services

1.4 Solution proposée

💡 Solution Retenue

MyHeart Healthcare System est une plateforme intégrée de gestion de santé basée sur une architecture microservices. Elle est composée de **six services indépendants**, d'un frontend React unifié et d'une infrastructure Docker complète permettant un déploiement simple et reproductible.

2 Partie I : Conception et Architecture

2.1 Identification des microservices

Le découpage en microservices repose sur le principe de *séparation des responsabilités*. Chaque service est autonome, possède sa propre base de données et communique via des API REST.

TABLE 1 – Identification et justification des microservices

Microservice	Responsabilité	Justification
Patient Service	Gestion des patients et utilisateurs	Référentiel central, indépendant
Appointment Service	Gestion des rendez-vous	Peut exister sans facturation
Pharmacy Service	Médicaments et prescriptions	Service autonome spécialisé
Lab Service	Analyses de laboratoire et résultats	Données non structurées (MongoDB)
EHR Service	Dossiers médicaux électroniques	Historique médical complexe
Billing Service	Facturation et paiements	Indépendant du calendrier

2.2 Architecture globale

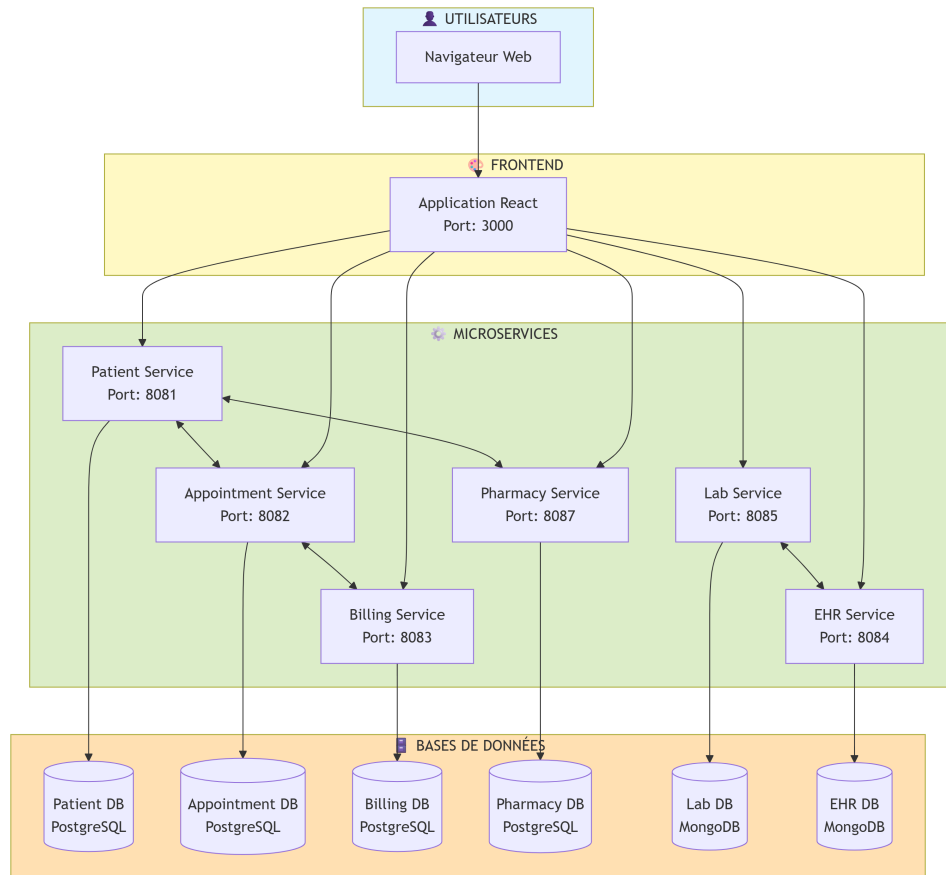


FIGURE 1 – Architecture globale du système MyHeart

2.3 Communication entre services

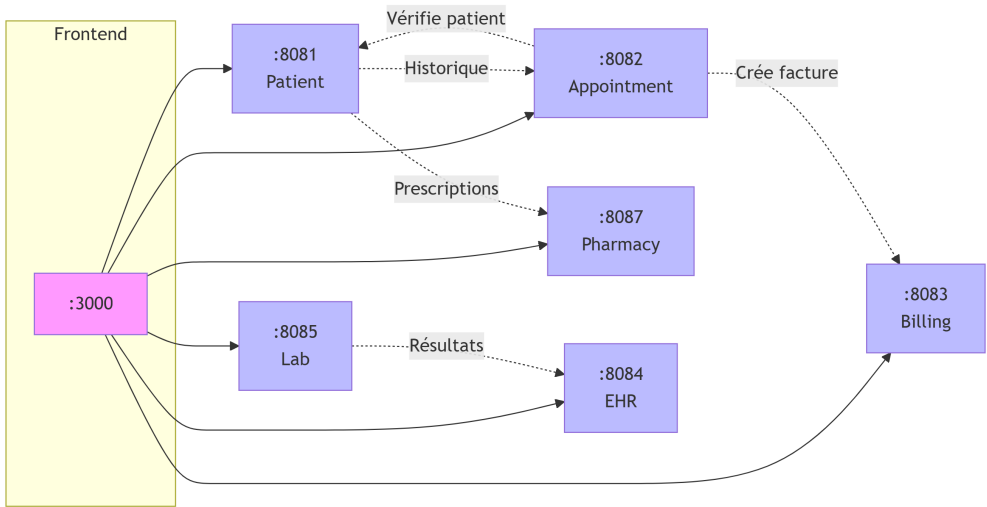


FIGURE 2 – Communication synchrone entre microservices via REST

2.4 Diagramme entité-relation (ERD)

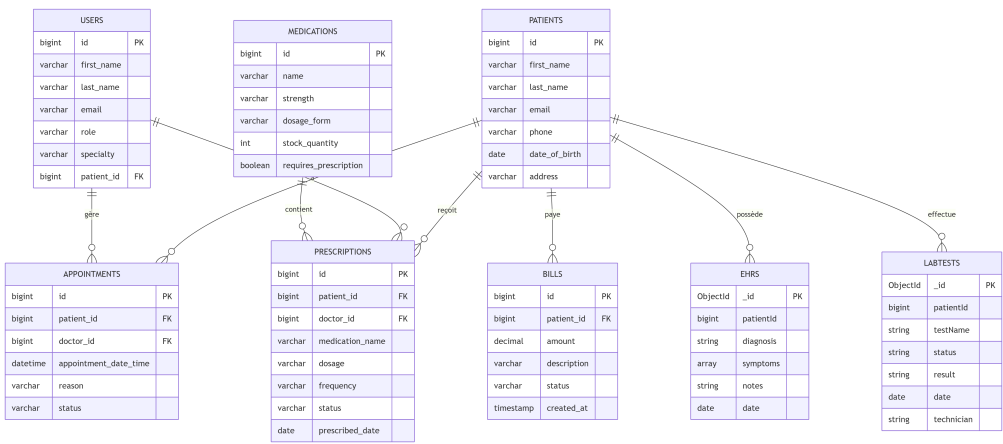


FIGURE 3 – Diagramme entité-relation (ERD) de la base de données

2.5 Structure des bases de données

2.5.1 Bases PostgreSQL (services Java / Spring Boot)

TABLE 2 – Bases de données PostgreSQL

Base de données	Port	Tables principales
patient-db	5432	patients, users
appointment-db	5433	appointments
pharmacy-db	5435	medications, prescriptions
billing-db	5434	bills

```

1 CREATE TABLE patients (
2     id                BIGSERIAL PRIMARY KEY,
3     first_name        VARCHAR(255) NOT NULL,
4     last_name         VARCHAR(255) NOT NULL,
5     email             VARCHAR(255) NOT NULL UNIQUE,
6     phone             VARCHAR(50),
7     address           TEXT,
8     date_of_birth     DATE,
9     social_security_number VARCHAR(50),
10    created_at        TIMESTAMP DEFAULT CURRENT_TIMESTAMP
11 );

```

Listing 1 – Schema SQL – Table patients

2.5.2 Bases MongoDB (services Node.js)

TABLE 3 – Bases de données MongoDB

Base de données	Port	Collections
ehr-db	27017	ehrs
lab-db	27018	labtests

```

1 {
2   "_id":      ObjectId("69a41b893ce6615bc53d4e75"),
3   "patientId": 1,
4   "diagnosis": "Suspicion d infection respiratoire aigue",
5   "symptoms":  ["fièvre modérée", "toux grasse",
6               "difficultés respiratoires"],
7   "notes":     "Prescription antibiotiques + kinesithérapie.",
8   "date":      ISODate("2026-03-01T10:57:13.264Z")
9 }

```

Listing 2 – Structure d'un document EHR (MongoDB)

2.6 Architecture de déploiement Docker

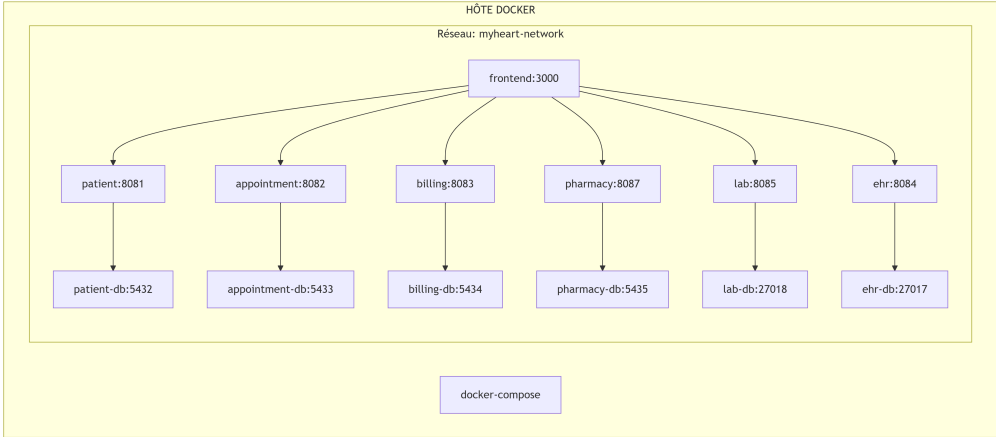


FIGURE 4 – Architecture de déploiement Docker Compose

2.7 Tableau récapitulatif des ports

TABLE 4 – Résumé des ports d'accès — Services et bases de données

Service / Base	Port	URL / Type
Services applicatifs		
Frontend	3000	http://localhost:3000
Patient Service	8081	http://localhost:8081
Appointment Service	8082	http://localhost:8082
Billing Service	8083	http://localhost:8083
EHR Service	8084	http://localhost:8084
Lab Service	8085	http://localhost:8085
Pharmacy Service	8087	http://localhost:8087
Bases de données		
patient-db	5432	PostgreSQL
appointment-db	5433	PostgreSQL
billing-db	5434	PostgreSQL
pharmacy-db	5435	PostgreSQL
ehr-db	27017	MongoDB
lab-db	27018	MongoDB

2.8 Stack technologique

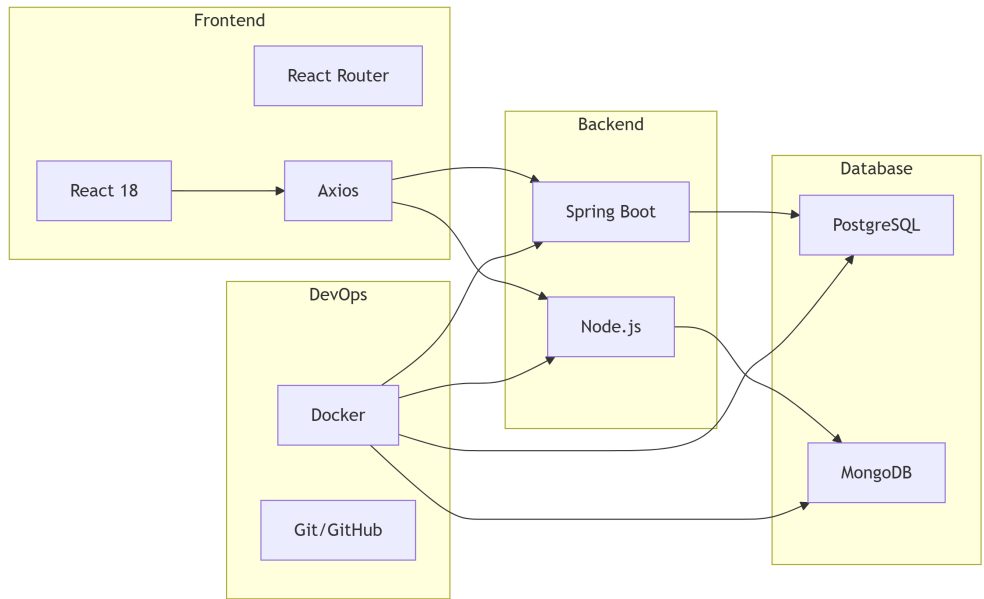


FIGURE 5 – Stack technologique du projet MyHeart

TABLE 5 – Technologies utilisées par domaine

Domaine	Technologies
Frontend	React 18, React Router v6, Axios
Backend Java	Spring Boot 3, Spring Data JPA, Hibernate
Backend Node.js	Express.js, Mongoose ODM
Bases de données	PostgreSQL 15, MongoDB 6
Conteneurisation	Docker, Docker Compose
Versionnement	Git, GitHub

3 Partie II : Implémentation et Déploiement

3.1 Flux de navigation dans l'application

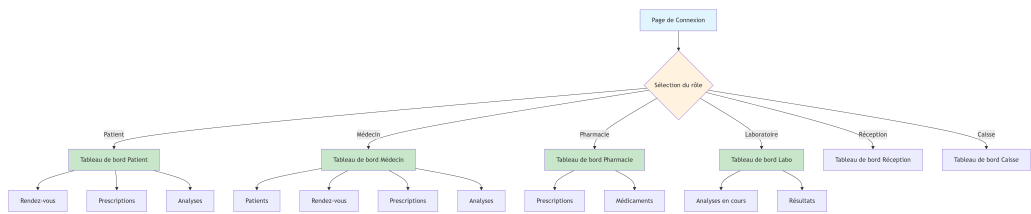


FIGURE 6 – Flux de navigation selon les rôles utilisateurs

3.2 Diagramme de séquence — Prise de rendez-vous

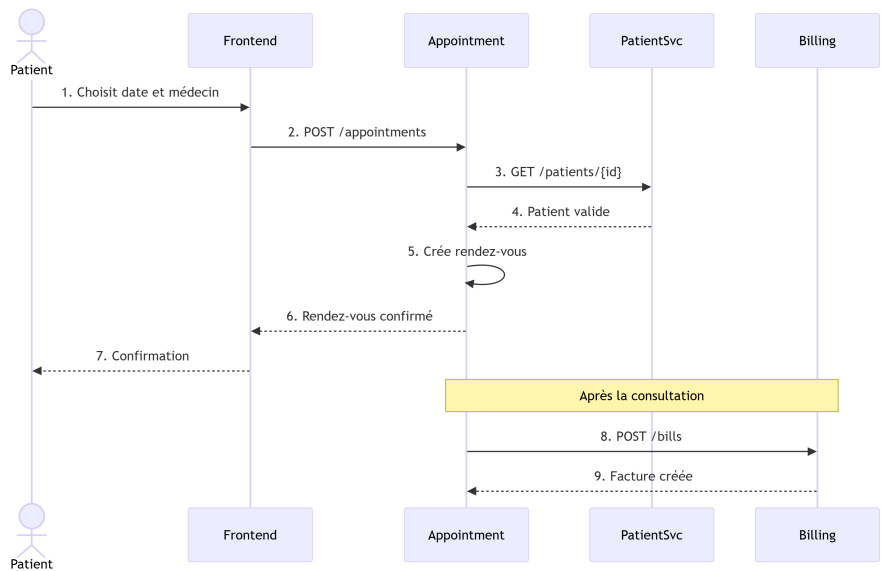


FIGURE 7 – Diagramme de séquence — Prise de rendez-vous et génération de facture

4 Interfaces Utilisateur

4.1 Page de connexion et sélection de rôle

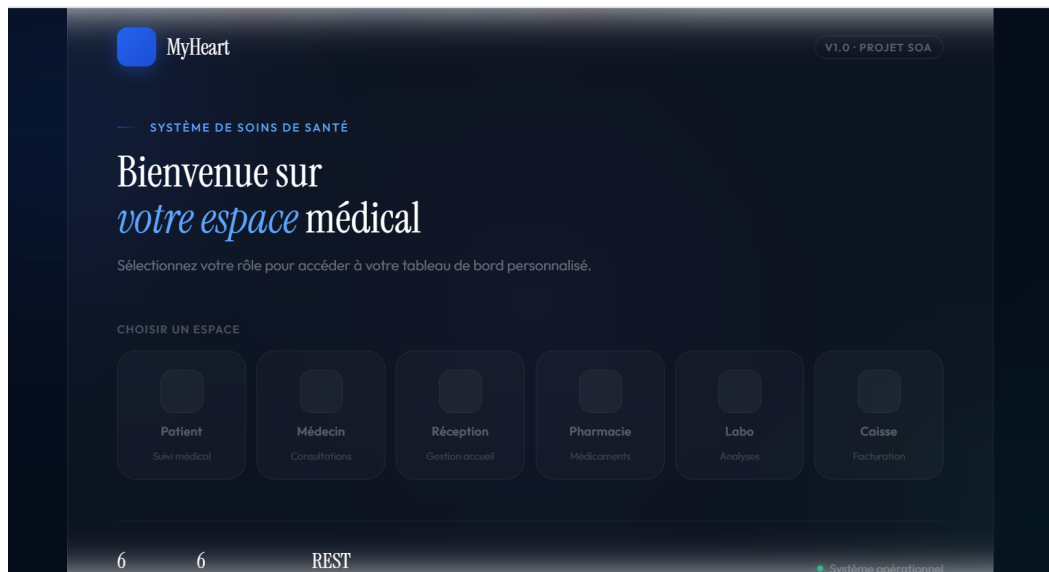
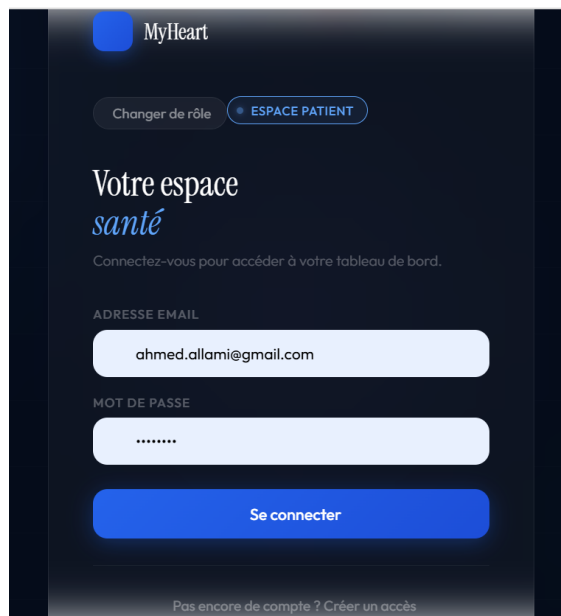
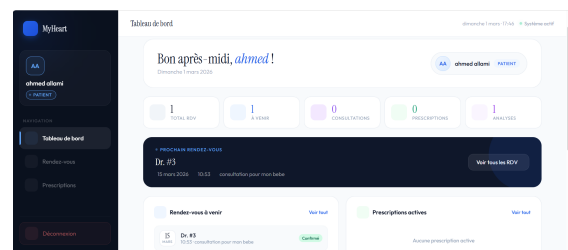


FIGURE 8 – Page de connexion et sélection de rôle

4.2 Espace Patient



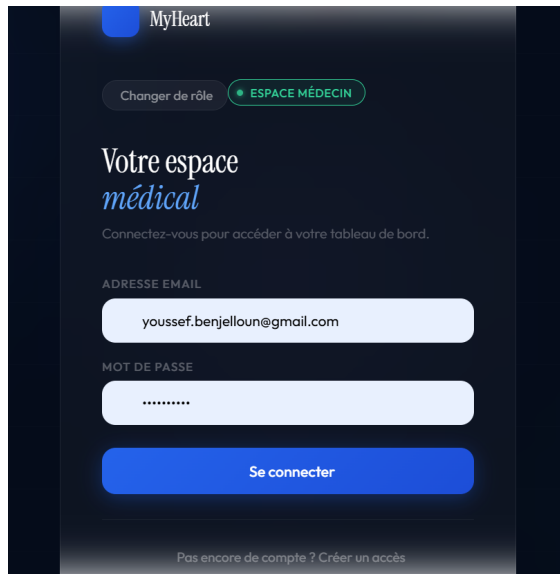
(a) Page de connexion — Patient



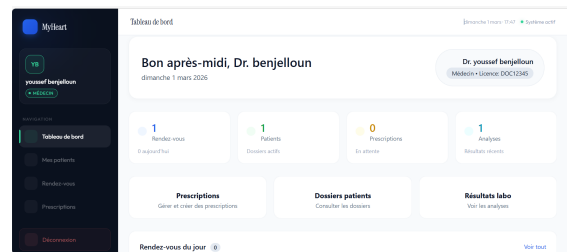
(b) Tableau de bord — Espace Patient

FIGURE 9 – Interfaces — Espace Patient

4.3 Espace Médecin



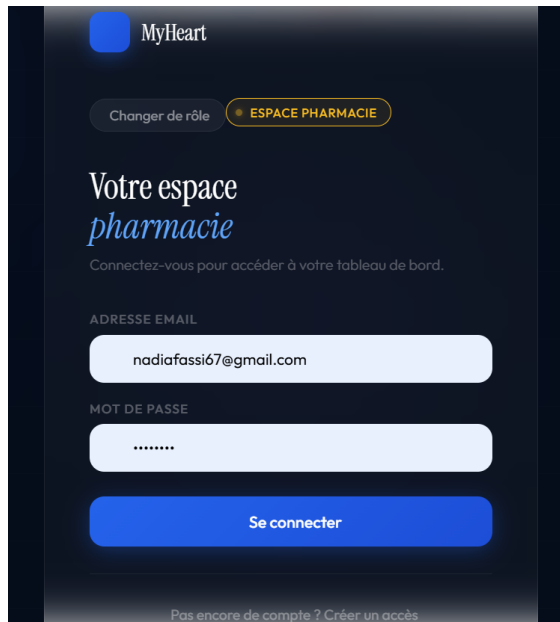
(a) Page de connexion — Médecin



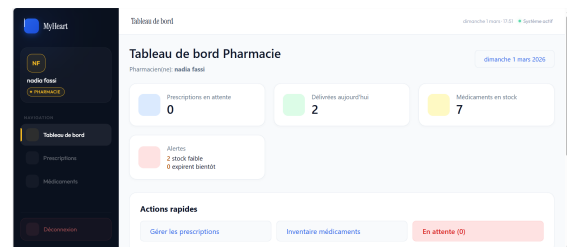
(b) Tableau de bord — Espace Médecin

FIGURE 10 – Interfaces — Espace Médecin

4.4 Espace Pharmacie



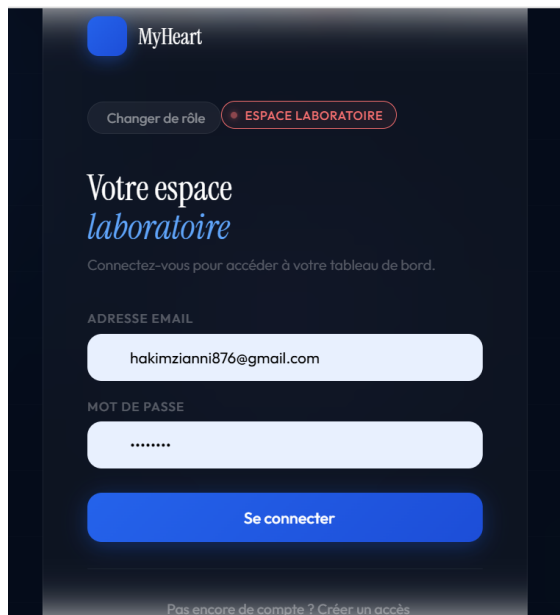
(a) Page de connexion — Pharmacie



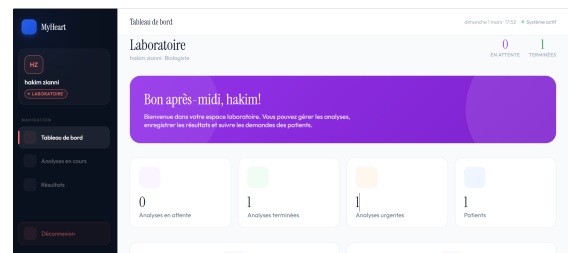
(b) Tableau de bord — Espace Pharmacie

FIGURE 11 – Interfaces — Espace Pharmacie

4.5 Espace Laboratoire



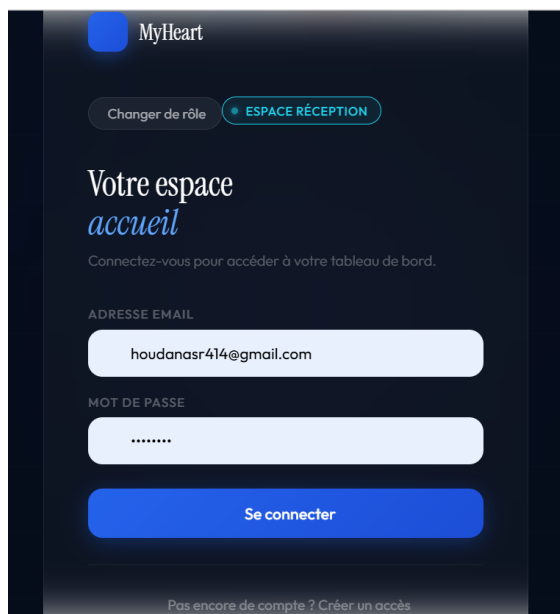
(a) Page de connexion — Laboratoire



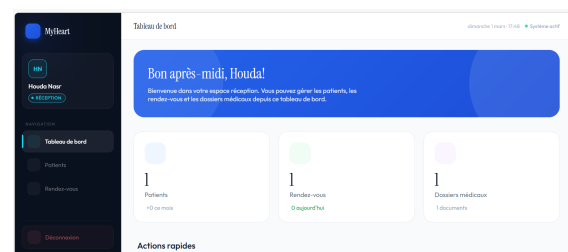
(b) Tableau de bord — Espace Laboratoire

FIGURE 12 – Interfaces — Espace Laboratoire

4.6 Espace Réception



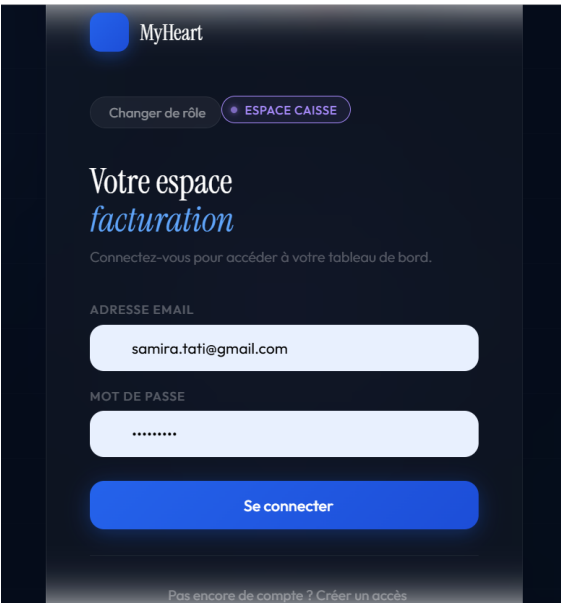
(a) Page de connexion — Réception



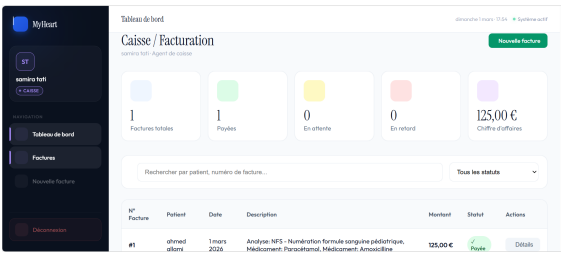
(b) Tableau de bord — Espace Réception

FIGURE 13 – Interfaces — Espace Réception

4.7 Espace Caisse



(a) Page de connexion — Caisse



(b) Tableau de bord — Espace Caisse

FIGURE 14 – Interfaces — Espace Caisse

5 Tests et Validation

5.1 Tests des API avec curl

```
1 $ curl http://localhost:8081/api/patients
2
3 [{
4   "id": 1,
5   "firstName": "ahmed",      "lastName": "allami",
6   "email": "ahmed.allami@gmail.com",
7   "phone": "07 44 55 66 77",
8   "address": "73 Rue Sebou, Agdal, Rabat 10080",
9   "dateOfBirth": "1999-11-19T00:00:00"
10  }]
```

Listing 3 – Test – Patient Service (GET /api/patients)

```
1 $ curl http://localhost:8082/api/appointments
2
3 [{
4   "id": 1,      "patientId": 1,      "doctorId": 3,
5   "reason": "consultation pour mon bebe",
6   "status": "CONFIRMED",
7   "appointmentDateTime": "2026-03-15T10:53:00",
8   "createdAt": "2026-03-01T10:54:12"
9  }]
```

Listing 4 – Test – Appointment Service (GET /api/appointments)

```
1 $ curl http://localhost:8087/api/medications
2
3 [{
4   "id": 1,  "name": "Doliprane",
5   "genericName": "Paracetamol", "strength": "1000mg",
6   "price": 2.50, "stockQuantity": 200,
7   "requiresPrescription": false
8 }, {
9   "id": 2,  "name": "Amoxicilline",
10  "strength": "500mg", "price": 4.75,
11  "stockQuantity": 149, "requiresPrescription": true
12 }]
```

Listing 5 – Test – Pharmacy Service (GET /api/medications)

```
1 $ curl http://localhost:8083/api/bills
2
3 [{
4   "id": 1,      "patientId": 1,
5   "amount": 125.00, "status": "PAID",
6   "paymentMethod": "CASH",
7   "description": "Analyse: NFS, Medicament: Paracetamol,
8   Amoxicilline",
9   "paidDate": "2026-03-01T12:44:05"
10  }]
```

Listing 6 – Test – Billing Service (GET /api/bills)

5.2 Vérification des bases de données

```

1 $ docker exec -it patient-db psql -U user -d patientdb \
2   -c "SELECT * FROM patients;"
3
4 id | first_name | last_name | email | phone
5 -----+-----+-----+-----+-----
6  1 | ahmed      | allami    | ahmed.allami@gmail.com | 07 44 55 66
7  77
8 (1 row)

```

Listing 7 – Vérification – PostgreSQL patient-db

```

1 $ docker exec -it pharmacy-db psql -U user -d pharmacydb -c "\dt"
2
3 List of relations
4 Schema | Name | Type | Owner
5 -----+-----+-----+-----
6 public | medications | table | user
7 public | prescriptions | table | user
8 (2 rows)

```

Listing 8 – Vérification – PostgreSQL pharmacy-db

```

1 $ docker exec -it ehr-db mongosh ehrdb \
2   --eval "db.ehrs.find().pretty()"
3
4 [{
5   _id: ObjectId('69a41b893ce6615bc53d4e75'),
6   patientId: 1,
7   diagnosis: "Suspicion d infection respiratoire aigue",
8   symptoms: ['fièvre modérée', 'toux grasse',
9             'difficultés respiratoires'],
10  notes: "Antibiotiques + kinésithérapie. Contrôle 48h.",
11  date: ISODate('2026-03-01T10:57:13.264Z')
12 }]

```

Listing 9 – Vérification – MongoDB ehr-db

```

1 $ docker exec -it lab-db mongosh labdb \
2   --eval "show collections"
3
4 labtests

```

Listing 10 – Vérification – MongoDB lab-db

6 Déploiement avec Docker

6.1 Fichier docker-compose.yml

```
1 services:
2   # --- Bases de données PostgreSQL ---
3   patient-db:
4     image: postgres:15
5     container_name: patient-db
6     environment:
7       POSTGRES_DB: patientdb
8       POSTGRES_USER: user
9       POSTGRES_PASSWORD: password
10    ports:
11      - "5432:5432"
12    volumes:
13      - patient-data:/var/lib/postgresql/data
14    networks:
15      - myheart-network
16    healthcheck:
17      test: ["CMD-SHELL", "pg_isready -U user -d patientdb"]
18      interval: 10s
19      timeout: 5s
20      retries: 5
21
22   appointment-db:
23     image: postgres:15
24     container_name: appointment-db
25     environment:
26       POSTGRES_DB: appointmentdb
27       POSTGRES_USER: user
28       POSTGRES_PASSWORD: password
29     ports:
30       - "5433:5432"
31     volumes:
32       - appointment-data:/var/lib/postgresql/data
33     networks:
34       - myheart-network
35     healthcheck:
36       test: ["CMD-SHELL", "pg_isready -U user -d appointmentdb"]
37       interval: 10s
38       timeout: 5s
39       retries: 5
40
41   billing-db:
42     image: postgres:15
43     container_name: billing-db
44     environment:
45       POSTGRES_DB: billingdb
46       POSTGRES_USER: user
47       POSTGRES_PASSWORD: password
48     ports:
49       - "5434:5432"
50     volumes:
51       - billing-data:/var/lib/postgresql/data
52     networks:
53       - myheart-network
54     healthcheck:
```

```
55     test: ["CMD-SHELL", "pg_isready -U user -d billingdb"]
56     interval: 10s
57     timeout: 5s
58     retries: 5
59
60     # --- NOUVEAU : Pharmacy Database ---
61     pharmacy-db:
62         image: postgres:15
63         container_name: pharmacy-db
64         environment:
65             POSTGRES_DB: pharmacydb
66             POSTGRES_USER: user
67             POSTGRES_PASSWORD: password
68         ports:
69             - "5435:5432"
70         volumes:
71             - pharmacy-data:/var/lib/postgresql/data
72         networks:
73             - myheart-network
74         healthcheck:
75             test: ["CMD-SHELL", "pg_isready -U user -d pharmacydb"]
76             interval: 10s
77             timeout: 5s
78             retries: 5
79
80     # --- Bases de données MongoDB ---
81     ehr-db:
82         image: mongo:6
83         container_name: ehr-db
84         ports:
85             - "27017:27017"
86         volumes:
87             - ehr-data:/data/db
88         networks:
89             - myheart-network
90         healthcheck:
91             test: echo 'db.runCommand("ping").ok' | mongosh localhost
92             :27017/test --quiet
93             interval: 10s
94             timeout: 5s
95             retries: 5
96
97     lab-db:
98         image: mongo:6
99         container_name: lab-db
100        ports:
101            - "27018:27017"
102        volumes:
103            - lab-data:/data/db
104        networks:
105            - myheart-network
106        healthcheck:
107            test: echo 'db.runCommand("ping").ok' | mongosh localhost
108            :27017/test --quiet
109            interval: 10s
110            timeout: 5s
111            retries: 5
```

```
111 # --- Microservices Spring Boot ---
112 patient-service:
113   build: ./backend/patient-service
114   container_name: patient-service
115   ports:
116     - "8081:8081"
117   environment:
118     SPRING_DATASOURCE_URL: jdbc:postgresql://patient-db:5432/
119     patientdb
120     SPRING_DATASOURCE_USERNAME: user
121     SPRING_DATASOURCE_PASSWORD: password
122   depends_on:
123     patient-db:
124       condition: service_healthy
125   networks:
126     - myheart-network
127
128 appointment-service:
129   build: ./backend/appointment-service
130   container_name: appointment-service
131   ports:
132     - "8082:8081"
133   environment:
134     SPRING_DATASOURCE_URL: jdbc:postgresql://appointment-db:5432/
135     appointmentdb
136     SPRING_DATASOURCE_USERNAME: user
137     SPRING_DATASOURCE_PASSWORD: password
138   depends_on:
139     appointment-db:
140       condition: service_healthy
141   networks:
142     - myheart-network
143
144 billing-service:
145   build: ./backend/billing-service
146   container_name: billing-service
147   ports:
148     - "8083:8081"
149   environment:
150     SPRING_DATASOURCE_URL: jdbc:postgresql://billing-db:5432/
151     billingdb
152     SPRING_DATASOURCE_USERNAME: user
153     SPRING_DATASOURCE_PASSWORD: password
154   depends_on:
155     billing-db:
156       condition: service_healthy
157   networks:
158     - myheart-network
159
160 # --- NOUVEAU : Pharmacy Service ---
161 pharmacy-service:
162   build: ./backend/pharmacy-service
163   container_name: pharmacy-service
164   ports:
165     - "8086:8086"
166   environment:
167     SPRING_DATASOURCE_URL: jdbc:postgresql://pharmacy-db:5432/
168     pharmacydb
```

```
165     SPRING_DATASOURCE_USERNAME: user
166     SPRING_DATASOURCE_PASSWORD: password
167     depends_on:
168       pharmacy-db:
169         condition: service_healthy
170     networks:
171       - myheart-network
172
173     # --- Microservices Node.js ---
174     ehr-service:
175       build: ./backend/ehr-service
176       container_name: ehr-service
177       ports:
178         - "8084:8084"
179       depends_on:
180         ehr-db:
181           condition: service_healthy
182       networks:
183         - myheart-network
184
185     lab-service:
186       build: ./backend/lab-service
187       container_name: lab-service
188       ports:
189         - "8085:8085"
190       depends_on:
191         lab-db:
192           condition: service_healthy
193       networks:
194         - myheart-network
195
196     networks:
197       myheart-network:
198         driver: bridge
199
200     volumes:
201       patient-data:
202       appointment-data:
203       billing-data:
204       pharmacy-data: # NOUVEAU volume
205       ehr-data:
206       lab-data:
```

Listing 11 – Fichier docker-compose.yml complet

6.2 Commandes de déploiement

```
1 # Lancer tous les services en arriere-plan
2 cd backend
3 docker-compose up -d
4
5 # Reconstruire les images avant de lancer
6 docker-compose up -d --build
7
8 # Afficher les conteneurs actifs
9 docker ps
10
```

```
11 # Consulter les logs d'un service
12 docker logs patient-service
13 docker logs -f appointment-service
14
15 # Arrêter et supprimer les conteneurs
16 docker-compose down
17
18 # Arrêter et supprimer aussi les volumes
19 docker-compose down -v
```

Listing 12 – Commandes Docker essentielles

7 Conclusion

7.1 Bilan du projet

Ce projet nous a permis de mettre en pratique les concepts fondamentaux d'architecture microservices et de SOA à travers le développement de **MyHeart Healthcare System**. Les principaux acquis sont les suivants :

- Conception et développement d'une architecture microservices avec **6 services indépendants**
- Implémentation des services backend avec **Spring Boot** (Java) et **Express.js** (Node.js)
- Développement d'une interface utilisateur unifiée avec **React 18**
- Mise en place de bases de données **PostgreSQL** (relationnel) et **MongoDB** (documents)
- Conteneurisation complète de l'application avec **Docker Compose**
- Gestion de la communication inter-services via des **API REST**

7.2 Difficultés rencontrées

- Gestion de la communication synchrone entre microservices (appels REST croisés)
- Coordination de bases de données hétérogènes (PostgreSQL + MongoDB)
- Configuration réseau et dépendances dans Docker Compose
- Synchronisation des données partagées entre services

7.3 Améliorations futures

- Ajout d'une **API Gateway** (ex. Spring Cloud Gateway, Kong)
- Mise en place d'un bus de messages asynchrone (**Kafka** ou RabbitMQ)
- Déploiement sur le cloud avec **Kubernetes** (GKE, EKS, AKS)
- Implémentation d'une authentification **JWT** / OAuth2
- Ajout de tests automatisés (unitaires, intégration, end-to-end)
- Mise en place d'une observabilité complète (Prometheus, Grafana, Jaeger)
- Mise en place d'un chat BOT

✔ Conclusion Générale

MyHeart Healthcare System démontre la faisabilité et les avantages d'une architecture microservices pour un système hospitalier : modularité, scalabilité et maintenabilité. Ce projet constitue une base solide pour des évolutions vers une plateforme cloud-native de production.

Annexes

A. Lien vers le dépôt GitHub

 <https://github.com/NASRHOUDA/myheart-healthcare-system.git>

B. Guide d'utilisation — Comptes de test

TABLE 6 – Comptes de démonstration pour l'application

Rôle	Email	Mot de passe
Médecin	youssef.benjelloun@gmail.com	youssef123
Patient	ahmed.allami@gmail.com	ahmed123
Pharmacie	nadiafassi67@gmail.com	nadia123
Laboratoire	hakimzianni876@gmail.com	hakim123
Réception	houdanasr414@gmail.com	houda123
Caisse	samira.tati@gmail.com	samira123

Avertissement sécurité

Ces comptes sont destinés uniquement à des fins de démonstration et d'évaluation. En production, les mots de passe doivent être hachés (bcrypt) et une authentification JWT doit être mise en place.